

Automated-review-rating-system

This project aims to automatically predicts the numerical ratings of the reviews given by the user. It uses Natural Language Processing (NLP), & Machine learning techniques for it.

Project overview and objective

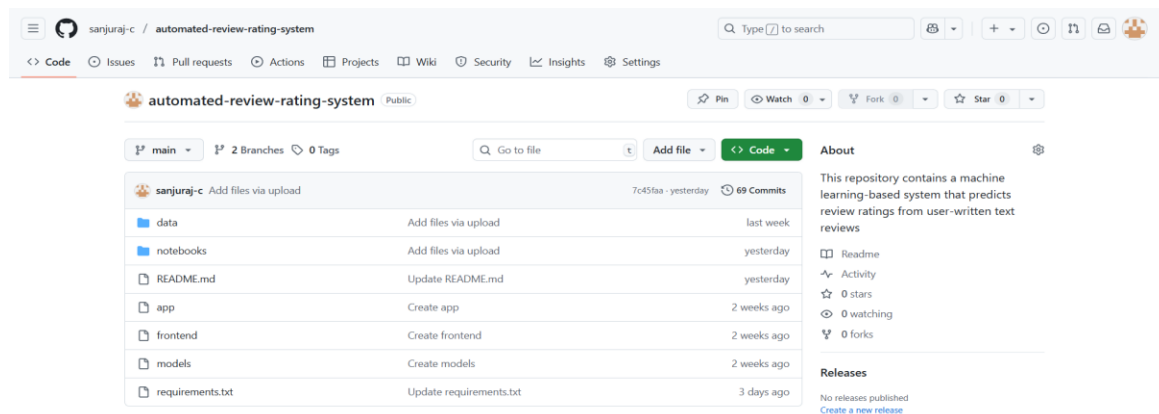
This project aims to build an automated review rating system that analyzes customer reviews and predicts the rating associated with each review. In many real-world scenarios, users provide detailed textual feedback, but numeric ratings may be missing, inconsistent, or biased. The goal is to train a machine learning model that can understand the sentiment and content of a review and accurately classify it into one of the standard rating categories (e.g., 1 to 5 stars).

By processing and analyzing review text using natural language processing techniques, this system helps businesses quickly identify the sentiment of customer feedback, monitor product performance, and make data-driven decisions without manual intervention.

Environmental Setup

The project was implemented using Python for data processing and analysis due to its rich ecosystem of libraries. Development and testing were carried out in Jupyter Notebook, providing an interactive environment for writing and visualizing code outputs. The entire codebase, along with dataset references and updates, was version-controlled using GitHub for collaboration and backup.

GitHub Project Setup:



The screenshot shows the GitHub repository page for 'sanjuraj-c / automated-review-rating-system'. The repository is public and has 69 commits. The file list includes:

File	Commit Message	Commit Time
data	Add files via upload	last week
notebooks	Add files via upload	yesterday
README.md	Update README.md	yesterday
app	Create app	2 weeks ago
frontend	Create frontend	2 weeks ago
models	Create models	2 weeks ago
requirements.txt	Update requirements.txt	3 days ago

The repository description states: 'This repository contains a machine learning-based system that predicts review ratings from user-written text reviews'. It also includes links to the README, Activity, Stars, Watching, and Forks sections.

Dataset description

To build the automated review rating system, data was gathered in two phases from various sources like Kaggle, Google search datasets, Data web scraping using API Key method in which, containing customer reviews along with corresponding ratings. The objective was to retain only the review text and rating fields after cleaning and preprocessing.

Phase 1 Datasets:

1. Amazon Echo Product Reviews (<https://www.kaggle.com/datasets/PromptCloudHQ/amazon-echo-dot-2-reviews-dataset>)

-This dataset contains reviews specifically for the Amazon Echo Dot 2, collected during September and October 2017. It includes detailed user feedback and metadata related to review authenticity and helpfulness. The complete version of this dataset is available via Data Stock, which contains reviews dating back to 2016.

-Key Fields: Page URL, Title, Review Text, Device Color, User Verified, Review Date, Useful Count, Configuration, Rating, Declaration Text.

2. Amazon Product Reviews (<https://www.kaggle.com/datasets/thedevastator/amazon-customer-reviews-with-2013-2019-sentiment>)

-This dataset contains reviews for a wide range of Amazon products such as smartphones, laptops, books, and refrigerators. Spanning six years, it also includes an in-house sentiment score based on the review content.

-Key Fields: Category, Review_Header, Review_text, Rating, Own_Rating

3. ChatGPT App Reviews (<https://www.kaggle.com/datasets/anandshaw2001/chatgpt-users-reviews>)

-This dataset comprises user reviews and ratings for the ChatGPT application. The reviews span several months and include timestamps, making the dataset suitable for temporal sentiment analysis.

-Key Fields: Review Id, Review, Ratings, Review Date.

4. Amazon General Reviews (<https://www.kaggle.com/datasets/dilekbarutu/amazon-review>)

-A comprehensive dataset containing user reviews for various products sold on Amazon. It includes essential metadata such as user IDs, timestamps, and helpfulness scores.

-Key Fields: reviewerID, asin, reviewerName, helpful, reviewText, overall, summary, unixReviewTime, reviewTime, day_diff

These cleaned datasets were combined to form Cleaned Dataset 1.

Phase 2 Datasets:

1. MakeupAlley Cosmetic Product Reviews (<https://www.kaggle.com/datasets/winston56/johnson-johnson-ogx-product-reviews>)

-These datasets contain reviews scraped from MakeupAlley and Ulta Beauty. Identifiable information has been removed. While the Ulta dataset includes accurate day-level timestamps, the MakeupAlley dataset includes only month and year.

-Key Fields: Date, Rating, Review, Product

2. Women's Clothing E-commerce Reviews (<https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews>)

-This dataset contains anonymized customer reviews for women's clothing products. It is suitable for NLP and recommendation system applications due to its structured features and diversity in product types.

-Included customer demographics and feedback information.

-Key Fields: Clothing ID, Age, Title, Review Text, Rating, Recommended IND, Positive Feedback Count, Division Name, Department Name, Class Name

These cleaned datasets were combined to form Cleaned Dataset 2.

Finally, both Cleaned Dataset 1 and Cleaned Dataset 2 were merged into a single dataset used for model training. Only reviews with meaningful content and valid rating labels were preserved.

Preprocessing Steps

After collecting and cleaning the datasets, the following initial preprocessing steps were applied to prepare the review text and rating data for model training:

1. Data Loading and Inspection

Each dataset was loaded using pandas and inspected using functions like `.head()`, `.tail()`, `.sample()`, `.shape`, `.columns`, `.info()`, and `.describe()` to understand structure and content.

```

#Load
df = pd.read_csv('data.csv')

#View
df.head()      # First 5 rows
df.tail()      # Last 5 rows
df.sample(3)   # Random 3 rows

#Inspect
df.shape       # Rows and columns
df.columns     # Column names
df.info()      # Data types and nulls
df.describe()  # Summary stats

```

2. Handling Missing Values

-Columns that had a large number of missing values or were irrelevant to the analysis were dropped.

-Rows with missing entries in the review text or rating columns were removed to maintain data quality.

```

df.isnull().sum()      # Count nulls
df.dropna(inplace=True) # Drop rows with any nulls
df['x'].fillna(df['x'].mean(), inplace=True) # Fill with mean

```

3. Dropping Irrelevant Columns

-Only the review text and rating columns were retained from each dataset.

-Other metadata like dates, user details, product configuration, etc., were considered unnecessary and discarded.

```

df.drop('x', axis=1, inplace=True) # Drop column

```

4. Duplicate Removal

-All datasets were checked for duplicate rows using `.duplicated()`.

-Any duplicate records found (if any) were removed to avoid data redundancy.

```

df.duplicated().sum() #finding duplicates
df.drop_duplicates(inplace=True) #removing duplicates

```

5. Data Merging

- Cleaned datasets from Phase 1 were combined into a single dataset.
- Similarly, Phase 2 datasets were cleaned and merged.
- Finally, both cleaned datasets were concatenated to form one unified dataset.

```
df_combined = pd.concat([df1, df2], axis=0) # Stack vertically
```

Review Text Processing

6. Emoji and Special Character Removal

- Emojis, symbols, and special characters were removed using regular expressions to focus only on meaningful textual content.

Remove emojis

```
text = re.sub(r'[^\w\s,]', '', text.encode('ascii', 'ignore').decode('ascii'))
```

Remove special characters and digits (keep letters and basic punctuation if needed)

```
text = re.sub(r'[^\A-Za-z\s]', '', text)
```

7. Stop Word Removal

- Common stop words (e.g., “the”, “is”, “and”) were removed using NLTK to reduce noise and retain only informative words.

#defining stopword removal function

```
stop_words = set(stopwords.words('english'))
```

```
punctuations = string.punctuation
```

```
def remove_stopwords(text):
```

```
    # Lowercase
```

```
    text = text.lower()
```

```
    # Remove punctuation
```

```
    text = ''.join([char for char in text if char not in punctuations])
```

```
    # Tokenize and remove stopwords
```

```
    words = text.split()
```

```
    filtered = [word for word in words if word not in stop_words]
```

```
    return ' '.join(filtered)
```

-stopwords used:

-From NLTK

-stop word count= 198

['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', 'aren't', 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'couldn', 'couldn't', 'd', 'did', 'didn', 'didn't', 'do', 'does', 'doesn', 'doesn't', 'doing', 'don', 'don't', 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'hadn', 'hadn't', 'has', 'hasn', 'hasn't', 'have', 'haven', 'haven't', 'having', 'he', 'he'd', 'he'll', 'he's', 'her', 'here', 'hers', 'herself', 'him', 'himself', 'his', 'how', 'i', 'i'd', 'i'll', 'i'm', 'i've', 'if', 'in', 'into', 'is', 'isn', 'isn't', 'it', 'it'd', 'it'll', 'it's', 'its', 'itself', 'just', 'll', 'm', 'ma', 'me', 'mightn', 'mightn't', 'more', 'most', 'mustn', 'mustn't', 'my', 'myself', 'needn', 'needn't', 'no', 'nor', 'not', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or', 'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 're', 's', 'same', 'shan', 'shan't', 'she', 'she'd', 'she'll', 'she's', 'should', 'should've', 'shouldn', 'shouldn't', 'so', 'some', 'such', 't', 'than', 'that', 'that'll', 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they', 'they'd', 'they'll', 'they're', 'they've', 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 've', 'very', 'was', 'wasn', 'wasn't', 'we', 'we'd', 'we'll', 'we're', 'we've', 'were', 'weren', 'weren't', 'what', 'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won', 'won't', 'wouldn', 'wouldn't', 'y', 'you', 'you'd', 'you'll', 'you're', 'you've', 'your', 'yours', 'yourself', 'yourselves']

8. Lemmatization

-Words were reduced to their root forms (e.g., “running” → “run”) using WordNetLemmatizer, improving consistency across similar word forms.

#defining lemmatization function

```
lemmatizer = WordNetLemmatizer()
```

```
def lemmatize_text(text):
```

```
    words = text.split()
```

```
    lemmatized = [lemmatizer.lemmatize(word) for word in words]
```

```
    return ' '.join(lemmatized)
```

-Why Lemmatization Over Stemming?

Lemmatization was chosen over stemming because it provides more accurate and meaningful word forms by reducing words to their dictionary base form (lemma), whereas stemming often results in incomplete or non-existent words. For example, "running" becomes "run" through lemmatization, but stemming might reduce it to "runn". This helps retain better context and grammatical correctness, which is especially important for tasks involving sentiment analysis and text understanding.

9. Filtering Out Short and Long Reviews

-Reviews with fewer than 3 words or more than 200 words were removed to retain only meaningful and focused feedback.

Keep only reviews that have word count between 3 and 200

```
filtered_df = df[df['Review'].apply(lambda x: 3 <= len(x.split()) <= 200)].reset_index(drop=True)
```

10. Conflicting Review Removal

-Reviews that did not align with their ratings (e.g., positive text with a negative rating or vice versa) were manually identified and removed.

#checking conflicting reviews
from textblob import TextBlob

Get sentiment polarity for each review (-1 = negative, 1 = positive)
df['Polarity'] = df['Review'].apply(lambda x: TextBlob(x).sentiment.polarity)

Optional: classify sentiment direction
def classify_sentiment(p):
 if p > 0.2:
 return 'positive'
 elif p < -0.2:
 return 'negative'
 else:
 return 'neutral'
df['Sentiment'] = df['Polarity'].apply(classify_sentiment)

Define conflict conditions
conflict_mask = (
 ((df['Rating'] >= 4) & (df['Sentiment'] == 'negative')) |
 ((df['Rating'] <= 2) & (df['Sentiment'] == 'positive'))
)

Get conflicting reviews for optional inspection
conflicting_reviews = df[conflict_mask]

Remove conflicting reviews
cleaned_df = df[~conflict_mask].reset_index(drop=True)

These preprocessing steps ensured a high-quality, cleaned dataset .

Visualizations Used

To better understand the data and guide modeling decisions, several visualizations were created during the exploratory data analysis (EDA) phase:

1. Histogram Plots of Rating Distributions

- Visualized how ratings were distributed across the dataset.
- Helped identify class imbalances (e.g., more 5-star reviews than 1-star).

2. Countplots (Per Rating Class)

- Displayed the number of reviews for each rating class using count plots.
- Useful for comparing rating frequency across different datasets.

3. Pie Charts of Overall Sentiment Distribution

- Ratings were grouped into broader sentiment categories like Positive, Neutral, and Negative.

- Pie charts helped visualize the proportion of each sentiment group in the dataset.

Before and After Cleaning/Balancing Comparisons

- Visualizations were also used to compare the dataset before and after cleaning and balancing. This helped in understanding how much the data was affected by preprocessing steps, especially in terms of class distribution. It clearly showed how the ratings became more evenly distributed after balancing, making the dataset suitable for training.

Final Combined Dataset

- It is the combination of both cleaned datasets of two phases.
- Total 6 different datasets are included in it.
- It has 86137 rows and 2 columns of 'Review Text' & 'Rating'.

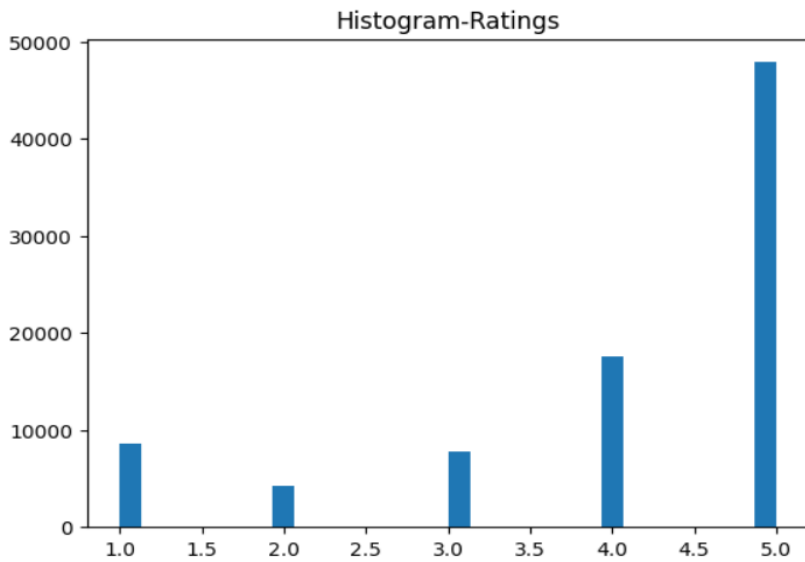
Done some visualization on the final combined data:

1.Histogram

```
plt.hist(final_combined_df['Rating'],bins=30)
```

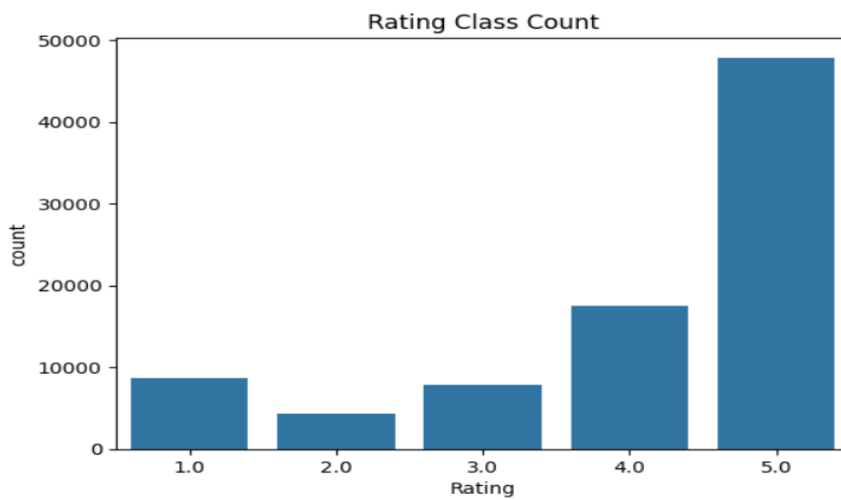
```
plt.title('Histogram-Ratings')
```

```
plt.show()
```

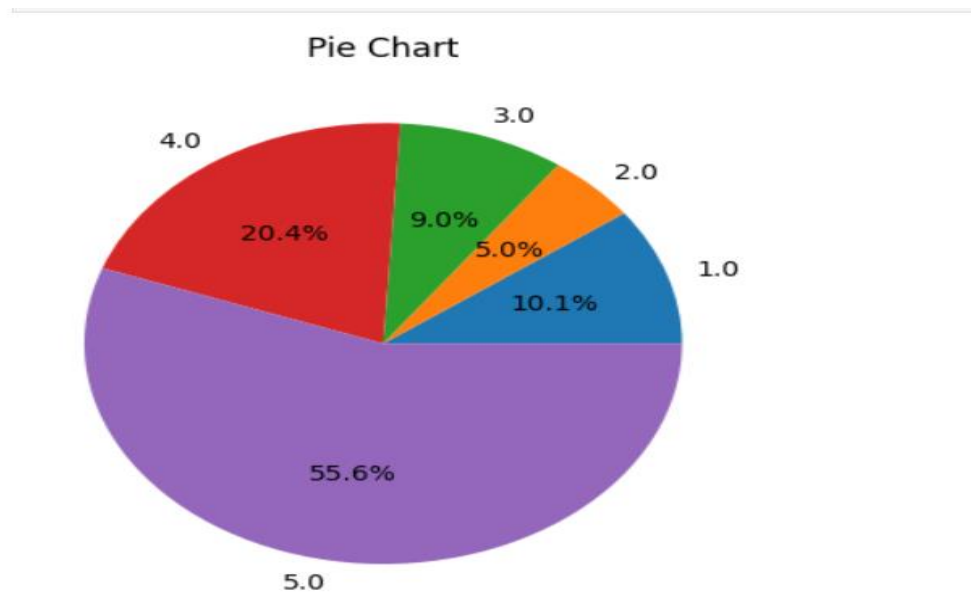
2.Count plot

```
sns.countplot(x='Rating', data=df2)  
plt.title("Rating Class Count")  
plt.show()
```



3. Pie Chart

```
rating_counts = df2['Rating'].value_counts().sort_index()  
plt.pie(rating_counts, labels=rating_counts.index, autopct='%1.1f%%')  
plt.title("Pie Chart")  
plt.show()
```



From the above visualizations, we can understand the rating data distribution. Rating 5 is higher in data and rating 2 is the lowest one.

Balanced Dataset

- Balanced dataset created from the final cleaned data, by loading it and again checked for any null values or duplicates for further clarification.
- Balanced it to a target count of 2700 at each rating class depending upon the availability of data in final cleaned one.

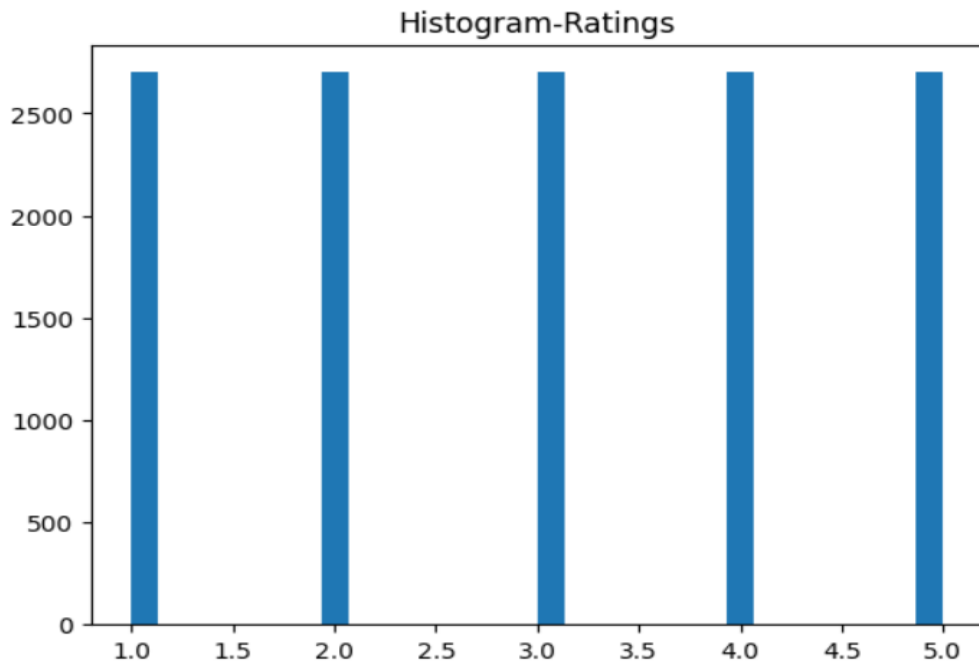
```
# Define target size
target_count = 2700

# Downsample each class
balanced_df_final = (
    filtered_df.groupby('Rating')
    .apply(lambda x: x.sample(n=target_count, random_state=42))
    .reset_index(drop=True)
)
```

Done some visualization on the Balanced data:

1.Histogram

```
plt.hist(balanced_df_final['Rating'],bins=30)
plt.title('Histogram-Ratings')
plt.show()
```



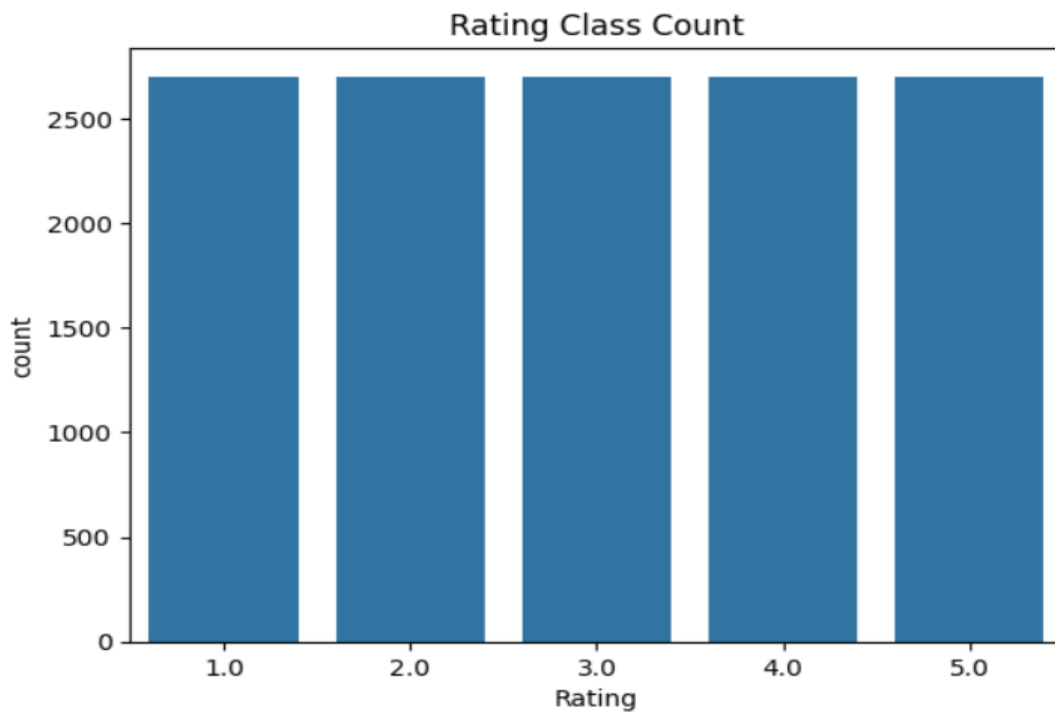
2.Countplot

#Count plot (for categorical data)

```
sns.countplot(x='Rating', data=balanced_df_final)
```

```
plt.title("Rating Class Count")
```

```
plt.show()
```



Sample Reviews Per Rating in Balanced Data:

```
for rating in sorted(balanced_df_final['Rating'].unique()):
    print(f"\n Rating: {int(rating)}")
    samples = balanced_df_final[balanced_df_final['Rating'] == rating].sample(3,
random_state=42)
    for i, review in enumerate(samples['Review'].values, 1):
        print(f"{i}. {review}\n")
```

output:

Rating: 1

1. poor camera build quality extremely disappointed
2. dont waste money another device already listen music one device time account course warning purchasing less useful hockey puck
3. cheapest quality paper useddont even think buy ithalf word visible

Rating: 2

1. hard time getting work keep getting dont know lot
2. kindly read carefully going purchase galaxy first big problem phone network issue heating problem battery back also expected equal mah compare mobile camera quality also good weak pixel quality processor also old working slow legging slow finger print sensor one thing good get proper notification voice quality also good annoying worst product samsung faith samsung type phone expected samsung cant compare phone brand r want return phone option refund purchasing replace cant get refund
3. although nothhing wrong speaker everything seems working fine gave star sound quality sound quality important come speaker speaker loud turning volume way still produce half sound quality lg bluetooth speaker would beneficial small room think speaker going adequate jam session sadly dissappointed feature voice command plus though although google bluetooth investigated purchasing

Rating: 3

1. found broken inside box
2. page less according content page actual page

3. shirt pretty color excited get however felt like fit flattering sort hung didnt much actually ended returning

Rating: 4

1. good mobile compare mi good internet connectivity connection like sim wifi product redmi

2. love anna suis line makeup everything dress beautiful usual sort way trying couldnt see anything spectacular made unique special somehow elastic waist make dress forgiving run big hem lovely scalloped detail nice put together dress wow justifies full price tag sale ill keep probably summer

3. battery seve lifetime

Rating: 5

1. really love watch earlier got fossil gen smartwatch unhappy experience mean battery would hardly last day compared experience samsung watch good enough battery life also liked rotatable bezel watch initially bit uncomfortable day two preferred use bezel build look watch elegant overall satisfactory experience watch

2. comfortable well made great legging skinny jean

3. gathered jersey dress beautiful flow right place however run large normally wear small many retailer dress however ordered x petite dress based review im glad fit perfectly dress flattering bustand rest dress hang flow nicely dressed statement jewelry depending occasion im happy purchase much bought one

Min/Max Review Length and Counts in Balanced Data

```
for rating in sorted(balanced_df_final['Rating'].unique()):  
    # Filter data by rating  
    subset = balanced_df_final[balanced_df_final['Rating'] == rating]  
  
    # Compute review lengths (word count)  
    lengths = subset['Review'].apply(lambda x: len(x.split()))  
  
    min_len = lengths.min()  
    max_len = lengths.max()  
  
    min_count = (lengths == min_len).sum()  
    max_count = (lengths == max_len).sum()
```

```
print(f"\n Rating: {int(rating)}")
print(f"  Min review length: {min_len} words (Count: {min_count})")
print(f"  Max review length: {max_len} words (Count: {max_count})")
```

Output:

Rating: 1
Min review length: 3 words (Count: 175)
Max review length: 199 words (Count: 1)

Rating: 2
Min review length: 3 words (Count: 119)
Max review length: 183 words (Count: 1)

Rating: 3
Min review length: 3 words (Count: 101)
Max review length: 177 words (Count: 1)

Rating: 4
Min review length: 3 words (Count: 133)
Max review length: 192 words (Count: 1)

Rating: 5
Min review length: 3 words (Count: 162)
Max review length: 191 words (Count: 1)

Train-Test-split data for Training

To evaluate the model's performance effectively, the dataset was split into two parts: 80% for training and 20% for testing. The training set was used to fit the model, while the test set helped in assessing its performance on unseen data. This split ensures better generalization and avoids overfitting.

```
from sklearn.model_selection import train_test_split

# Features and labels
X = balanced_df_final['Review']
y = balanced_df_final['Rating']

# Train-test split (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=42)
```

-Why Stratified Splitting?

Stratified splitting was used to ensure that the distribution of classes (ratings) remains consistent in both the training and testing sets. This is especially important when dealing with imbalanced datasets, as it helps maintain the proportion of each class across splits, leading to more reliable evaluation and better generalization of the model.

TF-IDF Vectorization

Since machine learning models cannot directly process raw text, vectorization was used to convert textual data into numerical format. In this project, the TF-IDF (Term Frequency–Inverse Document Frequency) method was applied, which transforms the text into a matrix that reflects how important a word is to a document in a collection. This approach not only captures the frequency of terms but also reduces the impact of commonly occurring words, helping the model focus on more informative terms for better predictions.

Term Definitions:

Term Frequency (TF):

This shows how often a word appears in a document.

$$TF = \frac{\text{Number of times the word appears in the document}}{\text{Total number of words in the document}}$$

Inverse Document Frequency (IDF):

This tells how unique or rare a word is across all documents.

$$IDF = \log \left(\frac{\text{Total number of documents}}{1 + \text{number of documents containing the word}} \right)$$

TF-IDF Score:

This is the final value used to represent each word. It is the product of TF and IDF.

$$TF - IDF = TF \times IDF$$

TF-IDF helps to give more importance to words that are unique and meaningful, and reduces the weight of common words.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize TF-IDF vectorize
vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1,2))

# Fit on training data and transform both train and test
X_train = vectorizer.fit_transform(X_train_text) # Fit on train
X_test = vectorizer.transform(X_test_text)      # Transform test
```


ML algorithm used- Logistic Regression

<https://github.com/sanjuraj-c/automated-review-rating-system/blob/main/ML%20Algorithm%20used%20for%20the%20model-balanced%20data.pdf>

Model Choice and Reason

- I used Logistic Regression for this project because it is a well-known algorithm for text classification and works well with TF-IDF features.
- It is also fast to train, supports multiclass classification, and is easy to understand.
- Compared to deep learning models, it is quicker and needs fewer resources.

Model Training Logic

1.Data Preparation – The cleaned review text was converted into numbers using TF-IDF vectorization so the model can work with it.

2.Splitting the Data – The dataset was split into 80% training and 20% testing using stratified sampling so that all rating classes were equally represented.

3.Model Settings –

- Algorithm: Logistic Regression
- Type: Multinomial (for multiple classes)
- Solver: LBFGS (works well for large datasets)
- Max Iterations: 1000 (to make sure the model fully trains)

4.Training – The model learned patterns from the training set.**5.Prediction** – The model was tested on unseen data to check how well it predicts ratings.

6.Hyperparameter Tuning – To improve the performance of the Logistic Regression model, hyperparameter tuning was performed using GridSearchCV. The main parameter tuned was the regularization parameter C, which controls the strength of regularization.

Evaluation results and interpretation

1. Accuracy: 47.40%

2. Precision/Recall/F1-score:

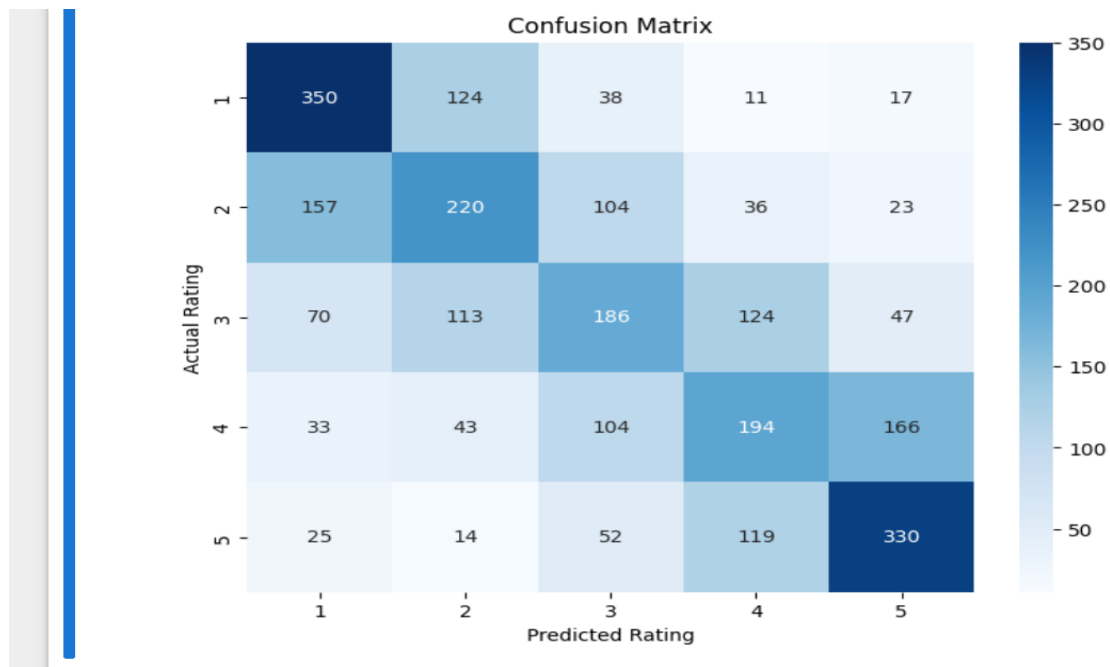
Classification Report:

	precision	recall	f1-score	support
1.0	0.551	0.648	0.596	540
2.0	0.428	0.407	0.417	540
3.0	0.384	0.344	0.363	540
4.0	0.401	0.359	0.379	540
5.0	0.566	0.611	0.588	540
accuracy			0.474	2700
macro avg	0.466	0.474	0.469	2700
weighted avg	0.466	0.474	0.469	2700

-Good performance for extreme ratings (1-star and 5-star).

-Mid-range ratings (2, 3, 4) were harder for the model to classify correctly.

3. Confusion matrix



From the confusion matrix:

- Many 1-star and 5-star reviews are predicted correctly.
- Mid-range ratings are often confused with nearby ratings (e.g., 3-star as 4-star).

This is common in review analysis because mid-level ratings have mixed sentiment.

